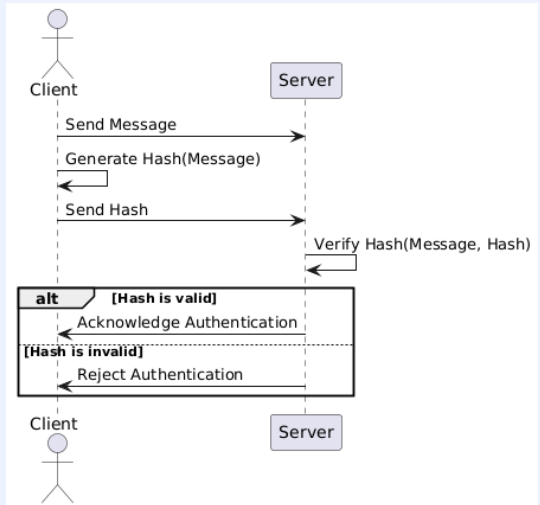


Message Authentication, Digital Signature and Applications of Cryptography

Unit III

Message Authentication

- Ensures the integrity and authenticity of a message.
- Protects against tampering and forgery.
- Achieved using:
 - Message Authentication Codes (MACs)
 - Cryptographic Hash Functions



Message Authentication Code (MAC)

- A MAC is a cryptographic checksum.
- Generated using a secret key and a message.
- Verification:
 - The receiver recalculates the MAC and compares it.
- Example Algorithms: HMAC, CMAC.

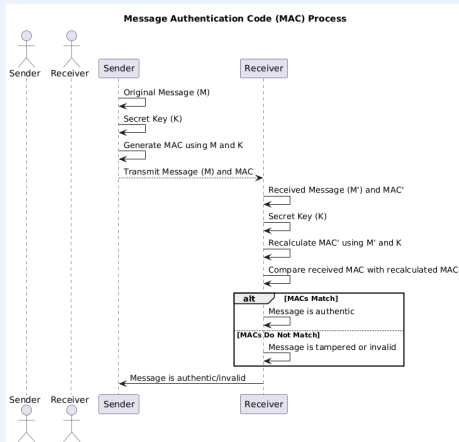


Figure: MAC Generation and Verification

Numerical Example: Checksum for Message Authentication

- Consider a message split into 4 blocks of 16 bits:

$$M_1 = 1101101010110011, M_2 = 1011010011001111,$$

$$M_3 = 1110001010111010, M_4 = 0111011100110101$$

- Compute the checksum:

- 1 Add the blocks using modulo- 2^{16} arithmetic:

$$\text{Sum} = M_1 + M_2 + M_3 + M_4$$

- 2 Compute the 1's complement of the sum:

$$\text{Checksum} = \text{NOT}(\text{Sum})$$

Example:

$$\begin{aligned} M_1 + M_2 &= 1101101010110011 + 1011010011001111 \\ &= 1100111110000010 \end{aligned}$$

Continue adding M_3 and M_4 .

Step-by-Step Binary Addition

Consider the following binary numbers:

$$M_1 = 1101101010110011, M_2 = 1011010011001111,$$

$$M_3 = 1110001010111010, M_4 = 0111011100110101$$

- ① Add the first two blocks ($M_1 + M_2$):

$$\begin{array}{r} 1101101010110011 \\ + 1011010011001111 \\ \hline 1100111110000010 \\ \text{(No carry-out in this step)} \end{array}$$

- ② Add the result to the third block ($M_1 + M_2 + M_3$):

$$\begin{array}{r} 1100111110000010 \\ + 1110001010111010 \\ \hline 1010100100111101 \quad \text{(Modulo-2}^{16} \text{ applied if carry-out occurs)} \end{array}$$

- ③ Add the result to the fourth block ($M_1 + M_2 + M_3 + M_4$):

$$\begin{array}{r} 1010100100111101 \\ + 0111011100110101 \\ \hline 0010000001110011 \quad \text{(Final modulo-2}^{16} \text{ sum)} \end{array}$$

Final Sum: 0010000001110011

Result: Checksum Calculation

- After all additions:

$$\text{Sum (mod } 2^{16}) = 0010000001110011$$

- Compute the 1's complement:

$$\text{Checksum} = 11011111011011001$$

- The sender transmits the original message blocks and the checksum.
- The receiver repeats the calculation to verify integrity.

Properties of Hash Functions

Deterministic

- A hash function is deterministic, meaning it will always produce the same output (hash) for a given input.
- Example: The SHA-256 hash of the message "Hello" will always be the same.

Efficient

- The hash function should be computationally efficient to compute, allowing fast processing of large amounts of data.
- Example: SHA-256 can be computed in a few milliseconds for small to medium-sized inputs.

Pre-image Resistance

- Given a hash value, it should be computationally infeasible to find the original input data.
- Example: Given the hash 'd2d2d2f5c33c...', it should not be possible to determine the original input.

Collision Resistance

- It should be difficult to find two distinct inputs that produce the same hash output.
- Example: SHA-256 is designed to resist collision attacks, where two different inputs produce the same hash.

Properties of Hash Functions

Small Changes in Input Produce Large Changes in Output

- A small change in the input should produce a significantly different hash value.
- Example: Changing one bit of the input to "Hello" results in a completely different hash value.

Fixed Output Length

- Regardless of the input size, the output hash will always have the same length.
- Example: SHA-256 produces a 256-bit hash, whether the input is a single character or a large file.

Properties of Hash Functions

Pre-image Resistance

- Pre-image resistance ensures that, given a hash value, it is computationally infeasible to find the original input that produced it.
- **Example:** If the hash of the string "Hello" is given as '2cf24dba5fb0a30e26e83b2ac5b9e29e1b170e3f55c9c8f2a57ec8209a29e1d1', it should be infeasible to determine the original input "Hello".
- **Goal:** Protects against attacks where an attacker tries to reverse-engineer the input from the hash.

Second Pre-image Resistance

- Second pre-image resistance ensures that, given an input and its hash, it is infeasible to find a different input that produces the same hash.
- **Example:** If the hash of "Hello" is '2cf24dba5fb0a30e26e83b2ac5b9e29e1b170e3f55c9c8f2a57ec8209a29e1d1', it should be infeasible to find another string (e.g., "World") that results in the same hash.
- **Goal:** Prevents an attacker from substituting a message with a different one that results in the same hash.

Collision Resistance

- Collision resistance ensures that it is infeasible to find two distinct inputs that produce the same hash output.
- **Example:** If you have two different inputs, "Hello" and "World", it should be infeasible to find a hash function collision where both inputs produce the same hash value.

Goal: Protects against attacks where an attacker might find two different messages with the same hash value

HMAC (Hash-based Message Authentication Code)

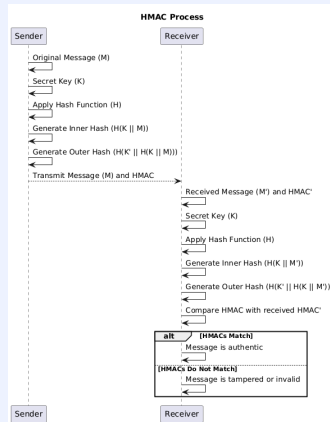
- HMAC is a construction for creating a message authentication code (MAC) based on a cryptographic hash function.
- It uses a secret key (K) along with a hash function (H) to provide authenticity and integrity.
- Steps to create HMAC:
 - 1 Apply the hash function to the message and secret key:

$$\text{Inner hash} = H(K \oplus \text{ipad} \parallel M)$$

- 2 Apply the hash function again to the result and the secret key:

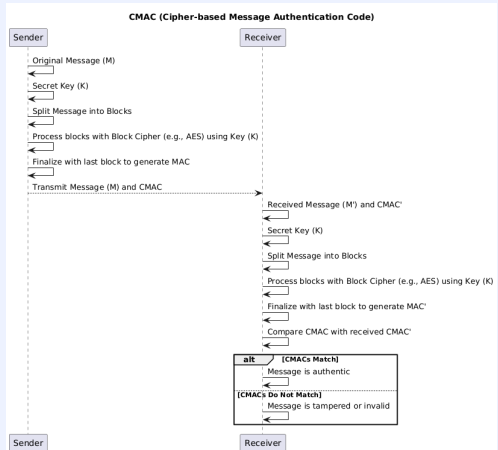
$$\text{HMAC} = H(K \oplus \text{opad} \parallel \text{Inner hash})$$

- Common Algorithms: HMAC-SHA256, HMAC-SHA1.



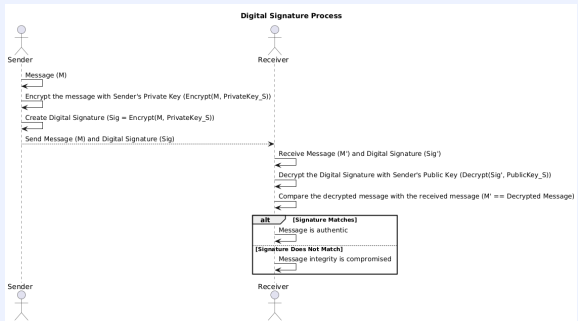
CMAC (Cipher-based Message Authentication Code)

- CMAC is a message authentication code (MAC) based on a block cipher.
- It provides data integrity and authenticity using a secret key (K) and a block cipher (e.g., AES).
- Steps to create CMAC:
 - 1 Divide the message into blocks.
 - 2 Process the blocks using the block cipher with the secret key to generate intermediate results.
 - 3 Final result is the MAC after processing all blocks.
- Common Algorithm: AES-CMAC, used with AES block cipher.



Digital Signatures

- Provide authenticity, integrity, and non-repudiation.
- Created using asymmetric cryptography.
- Process:
 - Encrypt the hash with the sender's private key.
- Example Algorithms: RSA, ECDSA.



RSA Digital Signature Algorithm

Steps for Signing a Message

- Alice wants to send a signed message to Bob.
- Alice has a message M , a private key d , and a public key e .
- The process of creating a digital signature involves:
 - ① Compute the hash of the message: $H(M)$.
 - ② Encrypt the hash with Alice's private key d to generate the signature:

$$\text{Signature} = \text{Sig} = (H(M))^d \mod n$$

Steps for Verifying the Signature

- Bob receives the message M and the signature Sig .
- Bob computes the hash of the received message: $H(M')$.
- Bob uses Alice's public key e to decrypt the signature:

$$\text{Decrypted Sig} = (\text{Sig})^e \mod n$$

- If $H(M') = \text{Decrypted Sig}$, the signature is valid.

RSA Digital Signature Algorithm

Numerical Example

Let's use a simple numerical example with small values for illustration:

- Choose a small modulus $n = 3233$ and exponents $e = 3$, $d = 7$.
- Assume Alice's message is $M = 123$.
- Compute the hash of the message: $H(M) = 123$.
- Signature:

$$\text{Sig} = 123^7 \mod 3233 = 865$$

- Bob receives the message $M' = 123$ and the signature $\text{Sig} = 865$.
- Bob computes the hash of the message: $H(M') = 123$.
- Bob decrypts the signature using Alice's public key:

$$\text{Decrypted Sig} = 865^3 \mod 3233 = 123$$

- Since $H(M') = \text{Decrypted Sig}$, the signature is valid.

Authenticity

- ## Non-Repudiation

- ## Integrity

- ## Additional Properties

- **Public Verification:** Anyone with the public key can verify the signature, ensuring open trust.
- **Resistance to Forgery:** The signature is computationally difficult to forge without the private key.
- **Efficient:** Digital signatures are computationally efficient, especially with asymmetric algorithms like RSA.

Applications of Cryptography

Data Security

- Encrypt sensitive data to prevent unauthorized access.
- Example: Using AES encryption to protect personal files on a computer.
- Ensures data confidentiality during storage and transmission.

Authentication

- Verify identities in systems and online platforms.
- Example: Two-factor authentication (2FA) using cryptographic tokens.
- Ensures that users are who they claim to be before granting access.

Digital Payments

- Secure online transactions using encryption.
- Example: Using SSL/TLS to secure online banking or shopping transactions.
- Protects financial data from interception and fraud.

Blockchain

- Ensure integrity and security of distributed ledgers.
- Example: Cryptographic hashing in Bitcoin to validate and secure transactions.
- Prevents tampering and fraud in decentralized systems.

Applications of Cryptography

Secure Communications

- Encrypt email, chat, and voice data.
- Example: Using PGP (Pretty Good Privacy) to encrypt email messages.
- Protects the privacy and confidentiality of communications.

Real-World Examples

HTTPS

- Secures web traffic using TLS/SSL.
- Example: HTTPS in web browsers ensures that communication between the client and server is encrypted.
- Protects data during transmission from eavesdropping and tampering.

E-Governance

- Digital signatures for document verification.
- Example: Government systems use digital signatures to authenticate official documents like tax filings or contracts.
- Ensures the authenticity and integrity of governmental records.

Bitcoin

- Uses cryptographic hashing and digital signatures.
- Example: Bitcoin uses SHA-256 for hashing and ECDSA for signing transactions.
- Ensures the integrity and authenticity of transactions on a decentralized network.

Secure Messaging

- End-to-end encryption in apps like WhatsApp.
- Example: WhatsApp uses Signal Protocol for secure messaging to ensure only the sender and receiver can read the messages.
- Protects user privacy from unauthorized access or surveillance.